TD9

Question 1 : Définition formelle des schémas de types (sans "..."):

$$\sigma ::= \tau \mid \forall \alpha . \sigma$$

Question 2 : Lesquelles sont correctes ?

1) Vrai $\tau \leqslant \tau$

2) Vrai on a remplacé α par num → num $\quad \alpha = num \to num$

$$\frac{\alpha \leqslant \alpha}{\forall \alpha . \alpha \leqslant \alpha \{num \to num / \alpha\}}$$

3) Vrai $\{num \to num / \alpha\}\ \{num / \beta\}$ $\quad \alpha = num \to num,\ \beta = num$

4) $\forall \beta . \alpha \to \beta \leqslant \alpha \to num$
$\leqslant \alpha \to (num \to num)$ on n'a pas généralisé. On n'a pas quantifié sur α
On peut avoir $\alpha \to num$
$\alpha \to (num \to num)$

5) Faux $\sigma \leqslant \tau$
$\forall \alpha . \sigma \leqslant \tau \{\tau' / \alpha\}$

On doit pas avoir de quantification à droite du $\leqslant$
On peut avoir dans τ quelconque qui a des quantifications dedans.

Question 3 : Expressions typables ? compléter annotations de types. Si un terme est bien typé, il termine une expression

1) $fun(f:?) \Rightarrow fun(g:?) \Rightarrow g(f\ 3)(f\ 12)$

$\to f : num \to \alpha$

$\to g : (num \to \alpha) \to (num \to \alpha)$

$\xrightarrow{fun} f : \vdash fun(g:) \Rightarrow g(f\ 3)(f\ 12) :$

$\xrightarrow{fun} f\ g \vdash g(f\ 3)(f\ 12) : B$

$\xrightarrow{APP} f\ g \vdash g(f\ 3) : \alpha \to 13\ f\ \ g \vdash (f\ 12) : \alpha$

$\to f : num \to \alpha,$
$g : \alpha \to num \to \beta \vdash g : \alpha \to \alpha \to \beta,\ f, g \vdash f\ 3 : \alpha$

$f : num \to \alpha,\ g \vdash f : num \to \alpha\ f\ g \vdash 12 : num$

$\to 1 : num \to \alpha \vdash f : num \to \alpha\ \ \Gamma \vdash 3 : num$

$\xrightarrow{VAR}$

2) $(fun(f:?) \Rightarrow f\ f)(fun(x:) \Rightarrow x\ x)$ PAS TYPABLE

$f : \alpha \to \beta \qquad f, x : 2$ types
α

$(\lambda x . x\ x)(\lambda y . y\ y)$ réduit

boucle $\Rightarrow$ termine pas $\Rightarrow$ non typable

Scanned with CamScanner

3) let comp : ? = fun (f : ?) ⇨ fun (g : ?) ⇨ fun (x : ?) ⇨ f (g x) in comp [?] (fun (x : ?) ⇨ x+1)
(fun (y : ?) ⇨ y) 12 TYPABLE
g : → x
f : →

comp [(num → num) → (num → num) → num → num)]

⇨ num (type finale

g : α → β
f : β → γ
x : α
comp : (β → γ) → (α → β) → α → γ
f = fun (x : num) ⇨ x+1 : num
g = fun (y : num) ⇨ y : num
x = 12 : num

4) (fun (f : ?) ⇨ (f 3) (f 12)) (fun (x : ?) ⇨ fun (y : ?) ⇨ y) PAS TYPABLE

(f 3) : α → β	f : num → α 3 : num	f : α → β → num → α
(f 12) : α	f : num → α	f : num → α → β

f 2 types différents.

on voit pas de x ailleurs ici, donc c'est β.

5) let f : ? (β) = fun (x : ? (β)) ⇨ fun (y : ?) ⇨ y in (f [?] 3) (f [?] 12) (α) TYPABLE

(f [num → (α → α) → (α → α)] 3) (f [num → α → α] 12) : α → α (type finale)

α ⇝ (α → α)
β ⇝ num

α ⇝ α
β ⇝ num

α ⇝ α
f : β → α → α
f : num → num
f : num → β
12 → 3 → 3

f est fun (int)
(bool)

Réduction :
β → (f [] 3) (fun (y : α) ⇨ y)
β → fun (y : α → α) ⇨ y) (fun (y : α) ⇨ y)
β → fun (y : α) ⇨ y

Scanned with CamScanner